UNIVERSIDADE ESTADUAL DE LONDRINA — DEPARTAMENTO DE COMPUTAÇÃO
2COP015: Inteligência Artificial (Prof. Dr. Antonio Parmezan) — Aluno Joao Vitor Ribeiro Flauzino

Estratégias de Busca Cega e Informada Aplicadas ao Problema Fruit Box

1. Introdução e Modelagem do Espaço de Estados

O problema abordado consiste no jogo combinatório Fruit Box(GameSaien), disposto em uma grade bidimensional $M \times N$ preenchida por maçãs com valores inteiros entre 1 e 9. A cada jogada, seleciona uma região retangular cujas maçãs somam exatamente 10, removendo-as do tabuleiro (suas células passam ao valor 0). Como células já vazias contam como zero e podem ser atravessadas por seleções futuras, a remoção de maçãs intermediárias desbloqueia novas possíveis soluções, tornando a ordem das decisões determinante e gerando ramificação combinatória e estados sem saída.

1.1 Representação dos estados, estado inicial e objetivo

Para representar um estado n, foi feito vetor linearizado da grade $G \in \{0, 1, \ldots, 9\}^{M \times N}$, onde $G[r, c] = 0$ indica célula vazia e $G[r, c] \in [1, 9]$ indica a maçã. O estado inicial n_0 é a grade original carregada com A_0 maçãs. O estado objetivo é qualquer estado em que todas as maçãs foram removidas ($\forall r, c,\ G[r, c] = 0$, soma residual igual a zero).

Operadores e custos: Um operador válido é definido por um retângulo $a = (r_1, c_1, r_2, c_2)$ com $0 \leq r_1 \leq r_2 < M$ e $0 \leq c_1 \leq c_2 < N$ tal que a soma das maçãs contidas em a seja exatamente 10; e cada uma das quatro bordas (r_1, r_2, c_1, c_2) contenha ao menos uma maçã, eliminando retângulos bordas vazias redundantes. Como cada ação remove soma 10, toda solução completa possui exatamente $d = S_0/10$ jogadas (onde S_0 é a soma inicial). Para diferenciar a qualidade das soluções quanto ao desperdício espacial, define-se o custo do passo como a área do retângulo arrastado, $c(n, a, n') = (r_2 - r_1 + 1) \times (c_2 - c_1 + 1)$, e o custo acumulado do caminho como $g(n) = \sum c(n_i, a_i, n_{i+1})$.

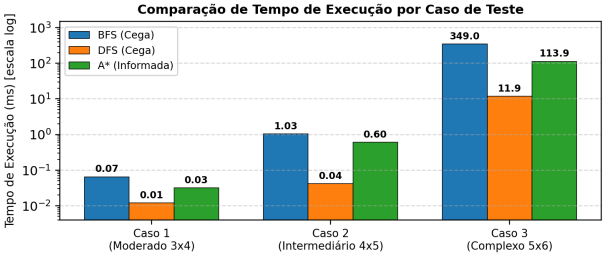
2. Estratégias de Busca Cega e Busca Informada (A*)

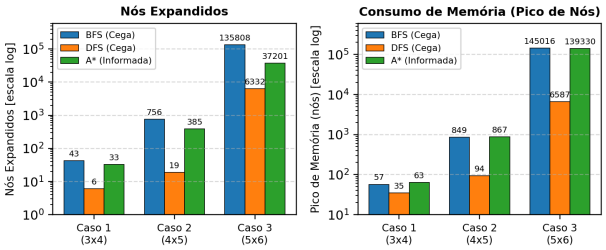
Busca não informada (BFS e DFS): Foi implementado a Busca em Largura (BFS) como estratégia cega principal, complementada pela Busca em Profundidade (DFS) com controle de estados visitados por tabela hash para um comparativo maior das soluções. A escolha da BFS justifica-se por o espaço de estados possuir profundidade finita $d = S_0/10$, garantindo completude. Entretanto, por expandir em camadas de profundidade e não segundo o custo acumulado $g(n)$, a BFS é ótima apenas em número de passos, retornando a primeira solução completa gerada na fila mesmo que seus retângulos atravessem células vazias desnecessárias. A DFS consome pouca memória ao descer por um único ramo até a folha, mas sofre com backtracking em becos sem saída e também não garante otimalidade de área. **Busca informada (A*):** Usando a busca Best-First segundo a abordagem A* com função de avaliação $f(n) = g(n) + h(n)$, tomando a heurística $h(n)$ como o número de maçãs ativas restantes no estado n. **Admissibilidade e consistência:** Uma jogada válida a que remove k maçãs ativas cobre no mínimo as k células dessas próprias maçãs (podendo cobrir adicionalmente $v \geq 0$ células já vazias). Logo, o custo do passo satisfaz $c(n, a, n') = k + v \geq k = h(n) - h(n')$, o que implica $h(n) \leq c(n, a, n') + h(n')$. Portanto, $h(n)$ é monotônica) e, como $h(\text{objetivo}) = 0$, também é admissível ($h(n) \leq h^*(n)$).

3. Casos de Teste e Resultados Experimentais

Foram definidos três casos de teste com dificuldade crescente, contendo escolhas entre cortes horizontais e verticais e jogadas ocultas desbloqueadas após remoções intermediárias: **(i) Moderado** (grade 3×4, 12 maçãs, soma 60, 6 jogadas) **(ii) Intermediário** (grade 4×5, 20 maçãs, soma 90, 9 jogadas, incluindo triplas verticais que liberam pares horizontais) e **(iii) Complexo** (grade 5×6, 30 maçãs, soma 140, 14 jogadas, com alta ramificação por permutação de ações independentes). A Tabela 1 resume o desempenho mediano com 5 execuções de cada estratégia:

Caso de Teste	Estratégia	Passos	Custo (g)	Nós Exp.	Memória (nós)	Tempo (ms)
Moderado (3×4)	BFS (Cega)	6	**18**	43	57	0,072
Moderado (3×4)	DFS (Cega)	6	**18**	6	35	0,013
Moderado (3×4)	A* (Informada)	6	**14**	33	63	0,035
Intermediário (4×5)	BFS (Cega)	9	**27**	756	849	1,08
Intermediário (4×5)	DFS (Cega)	9	**27**	19	94	0,040
Intermediário (4×5)	A* (Informada)	9	**21**	385	867	0,613
Complexo (5×6)	BFS (Cega)	14	**35**	135.808	145.016	411,85
Complexo (5×6)	DFS (Cega)	14	**35**	6.332	6.587	12,34
Complexo (5×6)	A* (Informada)	14	**31**	37.201	139.330	138,21





4. Discussão dos Resultados e Conclusão

Os experimentos demonstram como o crescimento do espaço de estados impacta de forma distinta cada estratégia. Na passagem do Caso 1 (3×4) ao Caso 3 (5×6), o número de subconjuntos de jogadas independentes (2^d) combinado às escolhas conflitantes entre retângulos horizontais e verticais faz a BFS saltar de 43 para **135.808 nós expandidos** (pico de 145.016 nós em memória e ~ 412 ms). Além disso, por explorar por ordem em largura sem ponderar a área dos retângulos, a BFS retorna soluções subótimas nos três casos (custos 18, 27 e 35). A DFS atinge uma folha rapidamente nos Casos 1 e 2, mas no Caso 3 sofre 6.332 expansões em backtracking e também converge para a solução subótima (custo 35). Em contraste, o **A*** encontrou a solução ótima de menor área nos três casos ($\boldsymbol{g^* = 14, 21}$ e $\mathbf{31}$) e reduziu drasticamente as expansões: no Caso 2 expandiu 49% menos nós que a BFS (385 vs. 756) e, no Caso 3, podou **72,6% do espaço explorado pela BFS** (37.201 vs. 135.808 nós), reduzindo o tempo para um terço (~ 138 ms).